

Análisis y Diseño con Patrones

Práctica 1: Principios de diseño, patrones de análisis y arquitectónicos (evaluada sobre 50 puntos)

Una empresa nos pide que diseñemos parte del software para gestionar los eventos que organizan en sus instalaciones. Los eventos pueden ser de tipo entretenimiento, conferencia o tipo taller de trabajo, dependiendo de las preferencias del cliente. Si el evento fuera de entretenimiento se puede elegir tipo de música a poner, y para la conferencia especificar al ponente que la dará. En sus instalaciones hay varias salas que pueden usar para un evento y estas salas se pueden distribuir de varias maneras: en forma de teatro, con todas las sillas mirando hacia un mismo sitio; en forma de cóctel, con mesas altas y sin muchas sillas; o en forma de banquete, con mesas y sillas.

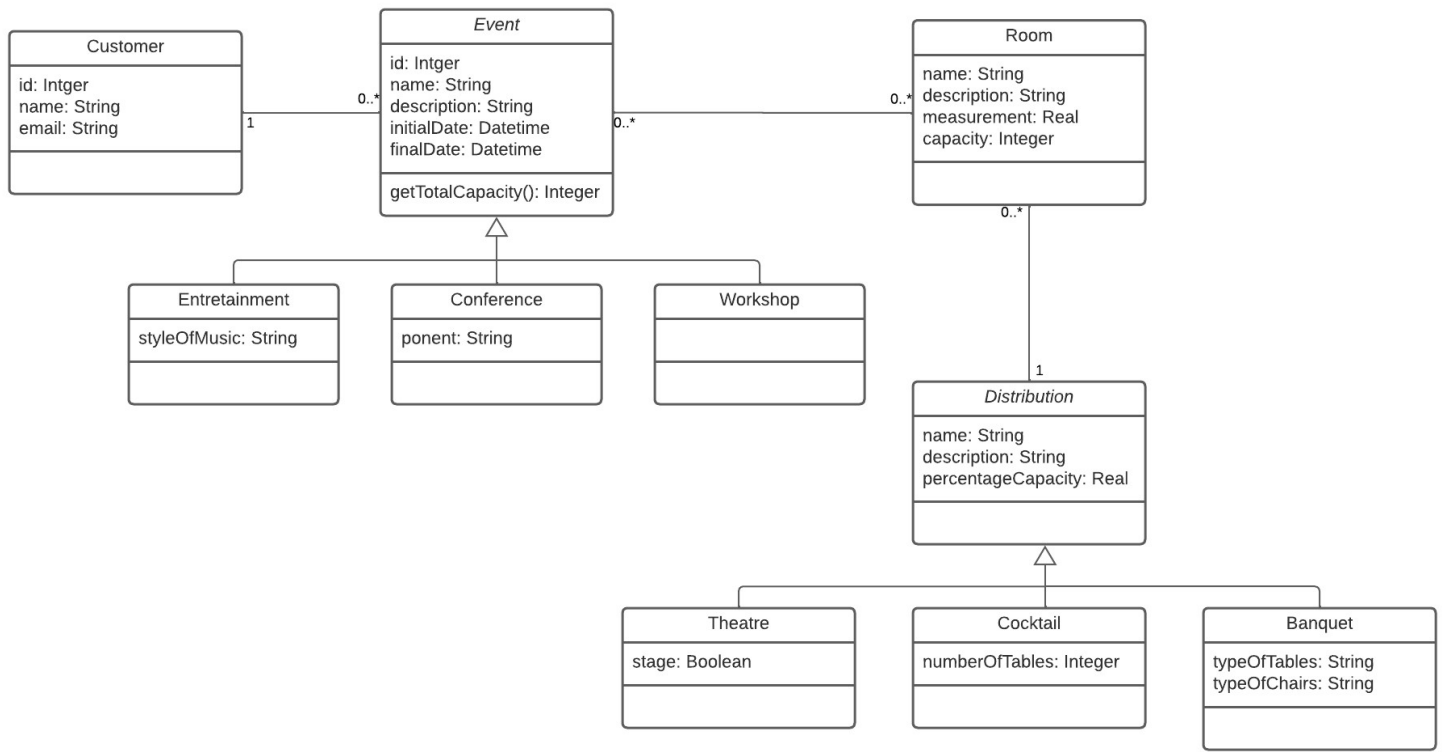
Cada sala tiene la medida en m² y la capacidad en número de personas, aunque dependiendo de la distribución de la sala la capacidad de esta cambia, ya que no es lo mismo tener a todas las personas de pie tipo cóctel que sentadas. Por ese motivo en la clase *distribución* tendremos el porcentaje de personas que caben, este porcentaje podría variar según la sala. Además, si la distribución es de tipo teatro se puede poner un mini escenario, lo que haría disminuir la capacidad de la sala en 10 personas (lo que ocupa ese escenario). Para la distribución tipo coctel se podrá especificar cuantas mesas tendremos, y para el tipo banquete se podrán elegir tipo de mesas y sillas.

Otra cosa a tener en cuenta es que las salas no pueden estar ocupadas por eventos que se solapen.

Este software es una parte de un sistema mayor. Obviaremos muchos elementos que un sistema real tendría. La razón de ello no es otra que la de hacer una práctica didáctica, y que la dificultad y dedicación prevista sean las deseadas.

Disponemos ya de un análisis previo que podemos utilizar como punto de partida, pero que habrá que corregir y mejorar. A continuación disponéis del diagrama de clases para esta parte del software.

Diagrama de clases:



Claves de las clases:

- Customer: id
- Event: id
- Room: name
- Distribution: name

Restricciones de integridad:

- La fecha de inicio de un evento debe ser anterior que la fecha final de este.
- Una sala no puede tener eventos solapados.
- Todos los integers y reales son números positivos.

Para calcular la capacidad total de un evento, nos han pedido diseñar la operación (`getTotalCapacity(): Integer`) de la clase `Event` que retorna la capacidad del evento en función de las salas que tenga reservadas y su distribución. A continuación se dispone del pseudocódigo de una posible implementación de esta operación:

```
public abstract class Event
```

```
{
    private Integer id;
    private String name;
    private String description;
    private Datetime initialDate;
    private Datetime finalDate;
    private Customer customer;
    private List<Room> rooms;

    public Integer getTotalCapacity()
    {
        Integer capacity = 0;
        foreach (Room r in rooms)
        {
            Integer roomCapacity = r.getCapacity();
            Distribution d = r.getDistribution();

            if(typeof(d)==Theatre)
            {
                Real percentCapacity = d.getPercentageCapacity()/100;
                capacity = capacity + (roomCapacity * percentCapacity);
                if(d.hasStage())
                {
                    capacity = capacity - 10
                }
            }
            else
            {
                Real percentCapacity = d.getPercentageCapacity()/100;
                capacity = capacity + (roomCapacity * percentCapacity);
            }
        }

        return capacity;
    }
}
```

Pregunta 1 (9 puntos)

Enunciado

Revisamos el pseudocódigo de la operación `getTotalCapacity()`: Integer de la clase Event y nos preocupa que no se cumplan algunos de los principios de diseño. En concreto, se pide:

- a) (3 puntos) Indica si este diseño satisface o no la Ley de Demeter y razona tu respuesta.
- b) (3 puntos) Indica si este diseño satisface o no el principio de diseño No Repetición y razona tu respuesta.
- c) (3 puntos) Indica si este diseño satisface o no el principio de diseño Abierto-Cerrado y razona tu respuesta.

Solución

- a) Esta solución no satisface la Ley de Demeter, ya que la clase Event, en el método `getTotalCapacity()`: Integer, utiliza las clases Distribution y Theatre que no tiene directamente asociadas. Es decir, obtiene instancias de Distribution y Theatre que no tiene asociadas, invoca métodos de las mismas y, por tanto, habla con desconocidos. En consecuencia, cualquier cambio en esta lógica propia de la clase Distribution o Theatre podría afectar al código de la clase Event.
- b) Esta solución no satisface el principio de No Repetición al tener código repetido en la misma función de `getTotalCapacity()`: Integer, dentro del if y el else hay código repetido y podría estar antes de las condiciones ya que comparten esa parte. Al hacer cualquier cambio siempre será más difícil de mantener.
- c) Esta solución viola el principio de diseño Abierto-Cerrado, ya que la clase Event es consciente de que existe una subclase (Theatre) de Distribution e invoca una operación. Si, más adelante, ampliamos el sistema para admitir una nueva subclase de Distribution y la operación `getTotalCapacity()`: Integer necesita invocar una operación de esta nueva subclase, deberíamos modificar el código de Event añadiendo una nueva condición para tratar esta nueva subclase.

Pregunta 2 (10 puntos)

Enunciado

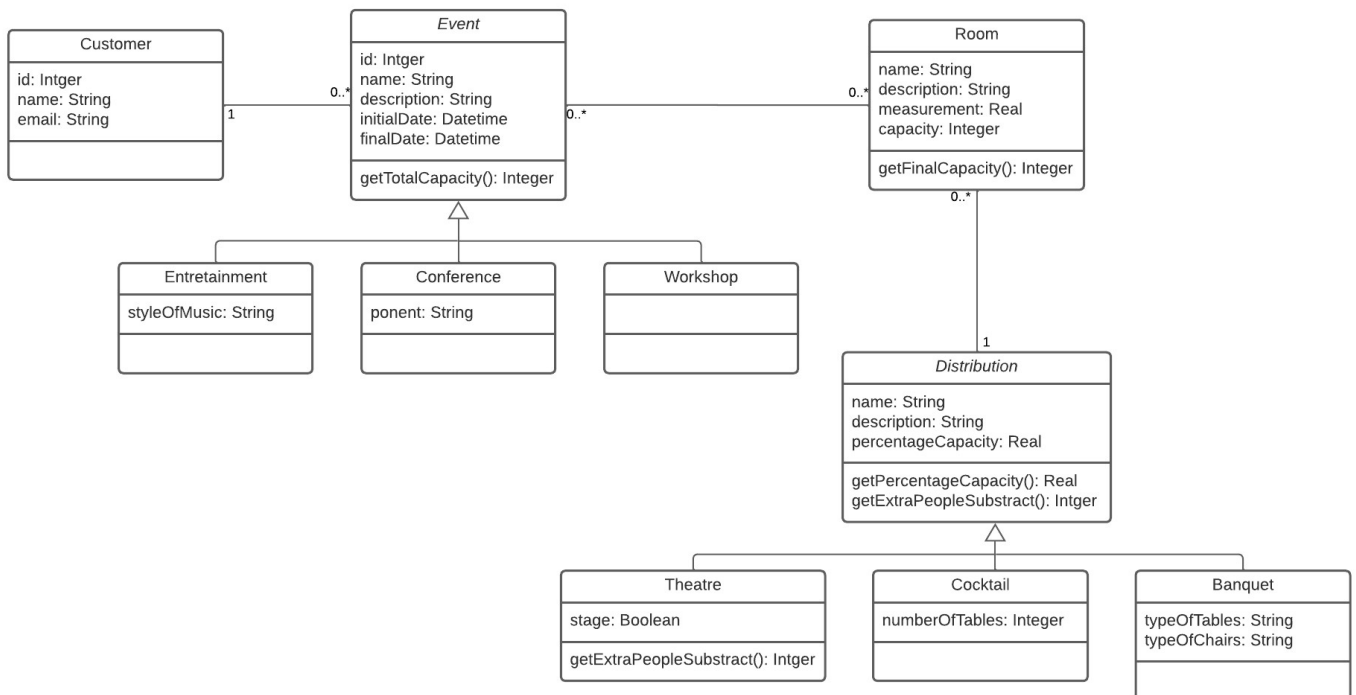
Propón una solución alternativa completa (también en pseudocódigo del mismo estilo) que corrija los problemas de la operación `getTotalCapacity()`: Integer de la clase Event que has detectado en la pregunta anterior y que, por tanto, cumpla los principios violados identificados en la pregunta 1. En concreto, se pide:

- a) (4 puntos) Modifica el diagrama de clases añadiendo todas las operaciones que creas necesarias para tu solución.

- b) (6 puntos) Pseudocódigo de la operación getTotalCapacity(): Integer de la clase Event cumpliendo todos los principios de diseño.

Solución

- a) Diagrama de clases con las operaciones:



- b) Pseudocódigo de la operación:

```

public abstract class Event
{
    private Integer id;
    private String name;
    private String description;
    private Datetime initialDate;
    private Datetime finalDate;
    private Customer customer;
    private List<Room> rooms;

    public Integer getTotalCapacity()
    {
        Integer capacity = 0;
        foreach (Room r in rooms)
    
```

```
        {
            Integer roomCapacity = r.getFinalCapacity();
            capacity = capacity + roomCapacity;
        }

        return capacity;
    }
}

public class Room
{
    private String name;
    private String description;
    private Real measurement;
    private Integer capacity;
    private Distribution distribution;

    public Integer getFinalCapacity()
    {
        Integer finalCapacity = 0;

        Real percentCapacity = distribution.getPercentageCapacity()/100;
        Integer extraPeopleSubstract = distribution.getExtraPeopleSubstract();

        finalCapacity = (capacity * percentCapacity) - extraPeopleSubstract;

        return finalCapacity;
    }
}

public abstract class Distribution
{
    private String name;
    private String description;
    private Real percentageCapacity;

    public Real getPercentageCapacity()
    {
        return percentageCapacity;
    }

    public Integer getExtraPeopleSubstract()
    {

```

```
        return 0;
    }

}

public class Theatre inherits Distribution
{
    private Boolean stage;

    public Integer getExtraPeopleSubstract()
    {
        if(stage)
        {
            return 10;
        }
        return 0;
    }
}
```

Pregunta 3 (6 puntos)

Enunciado

Muchos clientes han pedido a la empresa que les gustaría gestionar varios eventos en uno, es decir, tener un evento que se forme por distintos eventos. Por ejemplo, un evento que dura tres días está formado por tres eventos de tipo conferencia y dos de sesión de trabajo. Consiguiendo así integrar varios eventos en uno para gestionarlos mejor. Al ser una petición recurrente, la empresa nos propone buscar una solución que nos permita representar esta nueva opción de evento que llamaremos congreso en nuestro sistema (podéis partir del diagrama estático de análisis del enunciado). El tipo de evento congreso no puede contener otros congresos, solo eventos de los otros tipos.

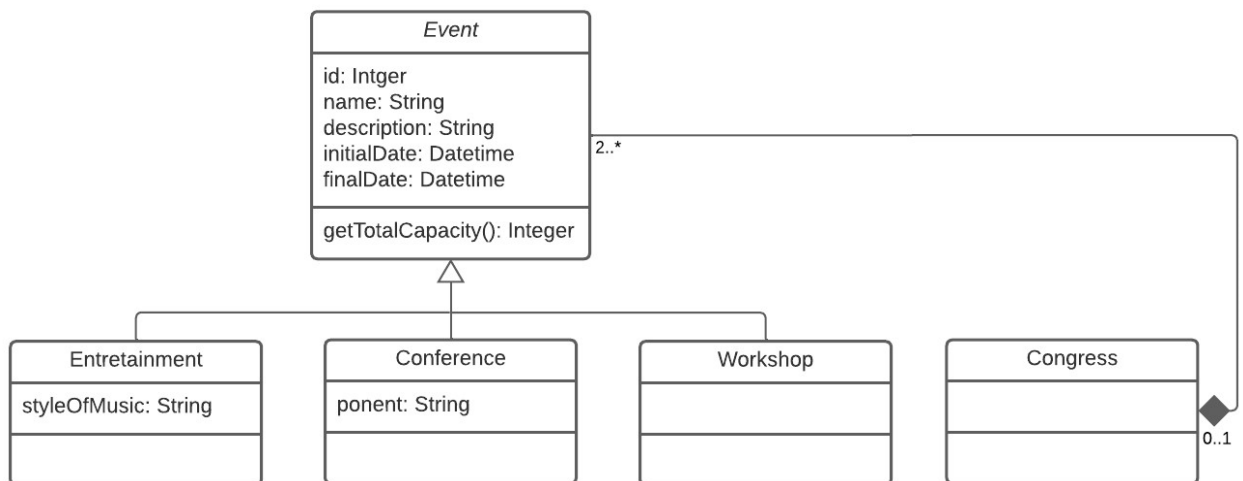
En concreto, se pide:

- (2 puntos) Aplica uno de los patrones de análisis que hay explicados en los materiales para representar esta información, explica cuál y razona su conveniencia.
- (4 puntos) Propón una solución detallada en forma de diagrama estático de análisis (incluyendo los nuevos atributos que aparezcan, las multiplicidades de las asociaciones y las

restricciones de integridad). Muestra solo las clases que cambien respecto al diagrama del enunciado.

Solución

- Para representar esta información aplicamos el patrón Objeto Compuesto. Los tipos de evento que ofrece la empresa son los que ya tenía y además añadiendo el tipo Congreso que agrupa varios eventos en uno.
- A continuación disponéis del fragmento del diagrama estático de análisis que representa la información de los diferentes tipos de distribución. Se han añadido también las restricciones de integridad.



Restricciones de integridad:

- La fecha initialDate debe ser menor a la finalDate
- Las fechas de los eventos que participan en el congreso tienen que estar dentro del rango de fechas del congreso
- Un congreso no puede contener eventos de tipo congreso

Pregunta 4 (6 puntos)

Enunciado

La empresa adquire un nuevo espacio fuera del país y se da cuenta que según el país donde esté los clientes no acaban de entender la medida de la sala, ya que está expresada en m2. Esta medida no es la usual en todos los países, se encuentran que tienen que ir traduciendo cada vez m2 a ft2 para los clientes anglosajones que emplean el sistema anglosajón de unidades.

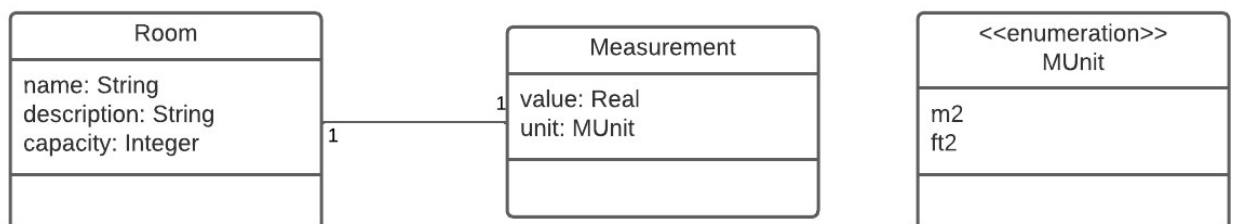
Al ser un potencial de clientes que va aumentando deciden poder representar la unidad de medida en el sistema que estamos diseñando (podéis partir del diagrama estático de análisis del enunciado).

En concreto, se pide:

- (2 puntos) Aplica uno de los patrones de análisis que hay explicados a los materiales para representar esta información, explica cual y razona su conveniencia.
- (4 puntos) Propón una solución detallada en forma de diagrama estático de análisis (incluyendo los nuevos atributos que aparezcan, las multiplicidades de las asociaciones y las restricciones de integridad). Muestra solo las clases que cambien respecto al diagrama del enunciado.

Solución

- Para representar esta información aplicamos el patrón Quantity. Las salas podrán tener distinta unidad de medida según lo requiera el cliente.
- A continuación disponéis del fragmento del diagrama estático de análisis que representa la información descrita en el enunciado. Se han añadido también las restricciones de integridad.



Restricciones de integridad:

- Value tiene que ser un valor mayor a 0.

Pregunta 5 (8 puntos)

Enunciado

Para que la empresa pueda utilizar este software necesita una aplicación web para acceder a los datos, introducir nuevos o modificar los existentes. Para definir esta capa de presentación de nuestro sistema, decidimos aplicar el patrón de arquitectura Modelo, Vista y Controlador (MVC).

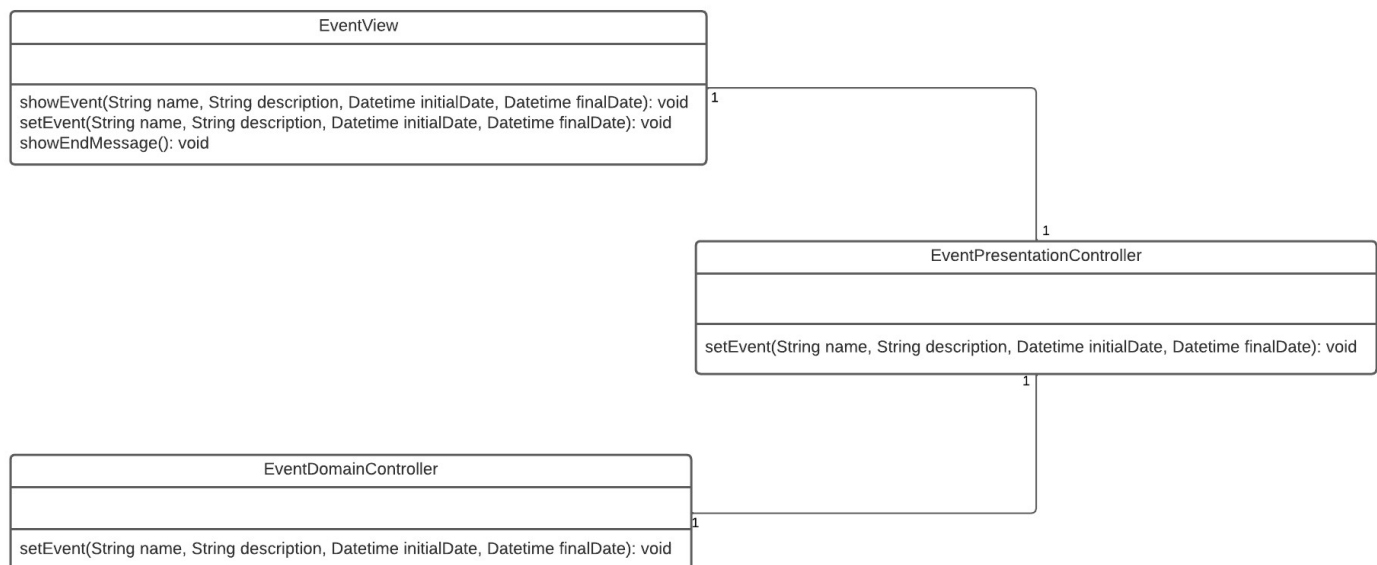
El caso de uso que se propone para este ejercicio es “Modificar descripción del Evento”. Este caso de uso permite a la empresa desde la pantalla de “Datos del Evento” modificar cualquiera de los atributos, guardarlo y que en la misma pantalla el sistema devuelva el mensaje “Successfully registered”.

Se pide:

- (5 puntos) Diagrama de clases del MVC para este caso de uso incluyendo las operaciones que sean necesarias (las operaciones tienen que contener los parámetros que necesiten y los tipos de retorno, si tienen) y una pequeña descripción de que hacen cada una de las operaciones.
- (3 puntos) Diagrama de secuencia o pseudocódigo del caso de uso, desde que se muestra por pantalla los datos de un evento para que la empresa pueda modificar cualquier dato hasta que el sistema le muestra el evento modificado y el mensaje “Successfully registered”.

Solución

a) Diagrama de clases



Operaciones de la clase EventView:

`EventView::showEvent(String name, String description, Datetime initialDate, Datetime finalDate)` muestra por pantalla los datos del evento (indicador en los parámetros de la operación).

`EventView::setEvent(String name, String description, Datetime initialDate, Datetime finalDate)` registra el evento que quiere modificar.

`EventView::showEndMessage()` muestra por pantalla el mensaje “Successfully registered”.

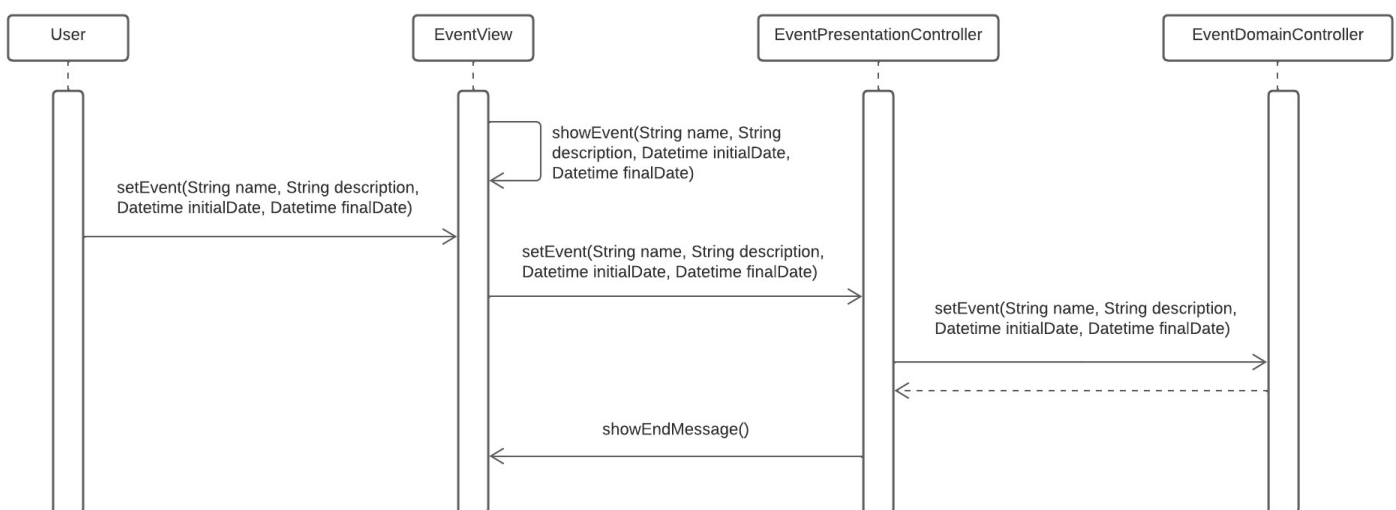
Operaciones de la clase `EventPresentationController`:

`EventPresentationController::setEvent(String name, String description, Datetime initialDate, Datetime finalDate)` detecta que el usuario ha seleccionado el registro de un evento y lo envía a la capa de dominio.

Operaciones de la clase `EventDomainController`:

`EventDomainController::setEvent(String name, String description, Datetime initialDate, Datetime finalDate)` registra las modificaciones del evento.

b) Diagrama de secuencia



Pregunta 6 (5 puntos)

Enunciado

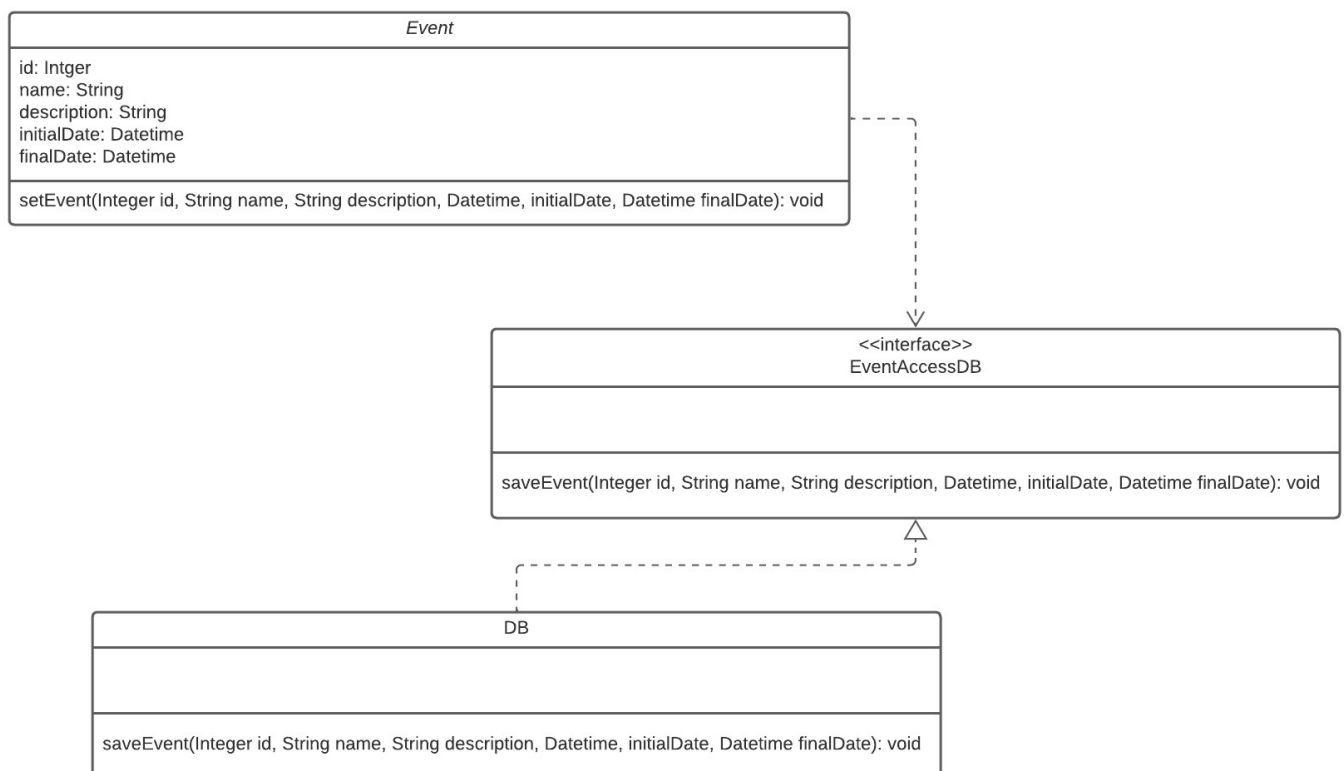
Seguimos diseñando la arquitectura del sistema, en el ejercicio anterior hemos utilizado el patrón Modelo-Vista-Controlador, que nos sirve para conectar la capa de Presentación con la de Dominio. Ahora queremos seguir diseñando ese caso de uso pero llegando hasta la capa de Servicios

técnicos (Capa de Datos), desde donde podemos acceder a la base de datos. Supongamos que tenemos una única base de datos y que queremos conectar la clase Event de nuestra capa de dominio con la base de datos para almacenar el cambio que se ha descrito en la pregunta anterior.

Se pide el diagrama de clases (correspondiente a la capa de dominio y a la capa de datos) para este caso de uso incluyendo las operaciones que sean necesarias (las operaciones tienen que contener los parámetros que necesiten y los tipos de retorno, si tienen). El diagrama de clases y las operaciones tienen que ser específicas para este caso de uso.

Solución

Diagrama de clases (la clase EventDomainController de la capa de dominio está en el diagrama de clases de la pregunta anterior)



Pregunta 7 (6 puntos)

Enunciado

Para acabar de diseñar la arquitectura en 3 capas del sistema, usando las tres capas clásicas de muchos sistemas de información:

1. Presentación
2. Dominio
3. Servicios técnicos (Datos)

Queremos reflexionar sobre a qué capa pertenece cada una de las clases (e interfaces, si hay) que han ido surgiendo en el enunciado (y a la solución que has elaborado hasta ahora):

- Clases del diagrama de clases del ejercicio 2.
 - Clases del diagrama de clases del ejercicio 5.
 - Clases del diagrama de clases del ejercicio 6.
- a) (2 puntos) Indica qué clases (e interfaces, si hay) pertenecen a la capa de servicios técnicos (en nuestro caso, capa de datos). Razona tu respuesta.
 - b) (2 puntos) Indica qué clases (e interfaces, si hay) pertenecen a la capa de dominio. Razona tu respuesta.
 - c) (2 puntos) Indica qué clases (e interfaces, si hay) pertenecen a la capa de presentación. Razona tu respuesta.

Solución

- a) Las clases de la capa de datos son aquellas que gestionan el acceso a la DB. En nuestro caso esta clase sería DB.
- b) Las clases de la capa de dominio es donde reside la lógica de negocio, donde trabajan el problema que nuestro sistema resuelve.
Esto incluye, naturalmente, las clases, las instancias de las cuales representan objetos, documentos o valores del mundo real:
 - Customer, Event, Entertainment, Conference, Workshop, Room, Distribution, Theatre, Cocktel, Banquet

También incluye las clases que ofrecen operaciones del nivel del dominio para ser usadas por la capa de presentación, que podemos llamar controladores de dominio:

- EventDomainController (de la solución del ejercicio 5)

Finalmente, las interfaces que representan el conjunto de objetos de dominio que el sistema conoce y que eventualmente serán implementadas por la capa de servicios técnicos para ofrecer persistencia:

- EventAccesDB (de la solución del ejercicio 6)
- c) Las clases de la capa de presentación son aquellas que gestionan la presentación de la información (vistas del MVC) y la recepción de los acontecimientos al sistema (controladores de presentación del MVC). En nuestro caso estas clases serían EventView y EventPresentationController.